BankOne Interview Questions and Answers

This note is based on the current BankOne backend and frontend code.

Architecture and stack

1. What is BankOne?
BankOne is a core banking platform with Spring Boot backend services, an Angular frontend, PostgreSQL persistence, JWT security, and Kafka-based notification handling.

2. What are the main application layers?
Controller, service, repository, entity, DTO, security, and frontend UI layers. Controllers expose REST APIs, services contain business logic, repositories handle persistence.

3. Why is Open Liberty used?
The backend is packaged as a WAR and runs on Open Liberty for local deployment and Java EE-style hosting. The same app can also run with embedded Boot for Docker/cloud.

4. What database pattern is used?
Spring Data JPA with PostgreSQL. Entities map to tables, and repositories provide query methods and pagination.

5. Why are DTOs used?
To keep API payloads separate from persistence entities and avoid leaking database structure to the frontend.

6. Why do repositories use interfaces only?
Spring Data generates implementations automatically from method names and specifications.

7. What is the role of `AuditableEntity`?
It provides shared `createdAt`, `updatedAt`, `createdBy`, `updatedBy`, and `version` fields for all auditable tables.

8. Why is optimistic locking useful here?
`@Version` helps prevent accidental overwrites when concurrent updates happen on the same record.

Authentication and authorization

9. How is login handled?
`POST /auth/login` delegates to `AuthenticationService`, which validates credentials and returns a JWT-based `LoginResponse`.

10. How does the app know the current user?
`GET /auth/me` returns the current authenticated user profile using the security context.

11. How are passwords changed?
`PUT /auth/password` accepts a validated request and updates the password through `AuthenticationService`.

12. How is role-based access enforced?
Backend methods use `@PreAuthorize`, and the Angular frontend uses route data plus `auth.hasAnyRole(...)` and route guards.

13. What roles exist in the system?
The code seeds roles like ADMIN, MANAGER, EMPLOYEE, TELLER, AUDITOR, and CUSTOMER through `RoleInitializer`.

14. Why use JWT instead of sessions?
JWT keeps the backend stateless, which fits API-based frontend/mobile access and simpler horizontal scaling.

15. What does `SecurityConfig` do?
It defines the security filter chain, CORS, JWT filter placement, stateless sessions, and role-based URL access.

16. Why is the JWT filter disabled as a servlet filter?
To avoid double registration. It must run only inside Spring Security's filter chain.

17. Why is `PasswordEncoder` BCrypt?
BCrypt is a secure adaptive hash for passwords and is the standard choice in Spring Security.

18. What does `LoginAttemptService` imply?
Failed logins are counted so accounts can be locked after repeated bad attempts.

19. What happens when an account is locked?
`AuthenticationService.login()` throws `LockedException`, and the global exception handler maps it to a response.

Customer and employee management

20. How are customers created?
`POST /customers` calls `CustomerService.createCustomer`, which saves the customer and then opens the first account.

21. What happens if a customer email or phone already exists?
The service checks uniqueness and throws a conflict error before insertion.

22. Why does customer creation also open an account?
This product assumes onboarding means customer creation plus initial banking access in one flow.

23. How are employees created?
`POST /users` is admin-only. `UserService.createUser` creates the user, assigns a role, and returns a response with a business employee code.

24. How are employee roles updated?
`PUT /users/{id}` updates the user record and switches the active role between ADMIN and EMPLOYEE access levels.

25. How are employee lists searched?
`GET /users` supports search, paging, and sorting via `PageRequests.of(...)` and `EmployeeSpecification`.

26. How is a business employee code generated?
The frontend and backend both use the same formatting convention through

`BusinessIdFormatter.employeeCode(...)`.

27. Why store both `User` and `UserRole`?
`User` stores identity and login data; `UserRole` stores role history and active/inactive role assignment.

28. What is the purpose of `RoleInitializer`?
It seeds default roles at startup so the system can assign roles without manual DB setup.

29. Why use `@PreAuthorize` on `UserController`?
Because user management is admin-only and should be blocked at the service endpoint.

30. What does the frontend do for employees?
It provides create/edit/list dialogs and uses role claims to hide admin-only features.

Accounts and policies

31. How are accounts opened?
`AccountService.openAccount` creates the account using the active account policy, generates an account number, and records an opening transaction if needed.

32. What is the account policy feature?
`/account-policies` lets the app create and fetch active policies by account type and currency.

33. Why does the code check policy effective dates?
To prevent using policies that are not yet valid or already expired.

34. Why does opening an account need `branchCode`?
Branch code is part of the account number generation and the business identity of the account.

35. What does `AccountNumberGenerator` solve?
It builds a business-safe account number from branch, type, currency, and ordinal.

36. Why keep `ordinal` in the account table?
It gives a stable sequence number for deterministic account numbering and business formatting.

37. What is the meaning of `availableBalance` vs `ledgerBalance`?
`availableBalance` is spendable balance; `ledgerBalance` tracks posted ledger state.

38. Why update both balances together?
The app keeps operational and ledger balances aligned for this banking model.

39. Why is account status stored as a string?
The entity stores the enum name, making DB rows easy to read and simplifying status transitions.

40. What does `updateAccountStatus` do?
It flips the status, sets closed/open timestamps when needed, and persists the change.

Transactions and money movement

41. How do deposit, withdraw, and transfer work?
They validate inputs, update balances, record transactions, and publish notification events.

42. Why are transfers locked in ascending account-id order?
To reduce deadlock risk when two accounts are updated in one transaction.

43. How are withdrawals protected from race conditions?
`findByIdForUpdate` locks the row before balance subtraction.

44. Why does deposit not lock the row the same way withdrawal does?
Withdrawal is the more sensitive insufficient-funds path; deposit is simpler but still transactional.

45. How are transactions stored?
Every money movement is persisted through `TransactionService.record(...)` into the transaction table.

46. Why is transaction recording separated from account mutation?
So the balance update and ledger insert are explicit business steps and easier to reuse.

47. Why is `TransactionType` an enum?
It keeps transaction categories controlled and avoids free-text values.

48. Why is `TransactionService.getByAccountId` read-only?
It only reads ledger history and should not participate in write transactions.

49. How are account and transaction lists paginated?
Controllers build `Pageable` objects using `PageRequests.of(...)` with allowed sort fields.

50. Why are allowed sort fields restricted?
To prevent invalid or unsafe sort values from the client.

51. Why does transfer write two transaction rows?
One debit row on the source account and one credit row on the destination account.

52. What should happen if transfer currency differs?
The service rejects it with a currency mismatch error because this banking model only supports same-currency transfer.

Notifications and Kafka

53. Why is Kafka used?
Kafka is used for asynchronous side effects, especially notification emails, so banking operations stay fast and transactional.

54. Is `app.kafka.notification-topic` a queue?
Functionally yes for this app, but technically it is a Kafka topic used as an event stream.

55. What does the notification publisher do?
`NotificationEventPublisher` creates a `BankActionEvent` and sends it to Kafka.

56. What does the notification consumer do?
`NotificationEventConsumer` listens to Kafka, composes an email, and sends it through the configured mail service.

57. Why should Kafka not wrap core balance updates?
Balance updates must remain synchronous and consistent. Kafka should handle only side effects like notifications, audit events, or downstream sync jobs.

58. Why is a consumer group used?
It allows listeners to scale and distribute work across instances while keeping one delivery path per partition assignment.

59. What is the risk in the current publisher?
The publisher logs Kafka failures but does not retry or persist failed events, so notifications can be lost.

60. What is the next reliability improvement?
Transactional outbox or retry/error-topic handling to avoid event loss.

Frontend

61. How is the frontend organized?
Angular standalone components are grouped by feature: auth, customers, employees, accounts, dashboard, profile, and management.

62. How does the frontend enforce roles?
The `Auth` service stores roles from login and helper methods like `hasAnyRole(...)` control route access and UI visibility.

63. Where is the API base URL configured?
In `src/app/core/config/api-config.ts`, which reads from the environment files.

64. How are API failures shown to the user?
Dialogs and pages use a shared `apiErrorMessage(...)` utility plus notification toasts.

65. What frontend pattern is used for lists?
Signals + RxJS streams (`toSignal`, `combineLatest`, `switchMap`) drive search, paging, sorting, and loading state.

66. Why do customer and account lists use signals?
To keep UI state reactive without manual subscription cleanup.

67. How do list pagination components work?
They translate page index/size into API paging parameters and expose controls like first/last/jump.

68. Why is the management page important?
It's the admin hub for creating customers and staff and for future admin utilities.

69. How are dialogs used in the UI?
Create/edit actions open Material dialogs, and the closing result triggers refresh/navigation.

70. Why does the UI show business codes like C00001 / E00001?
They're friendlier than raw numeric ids and match business-facing workflows.

Deployment and runtime

71. Where are logs written on Open Liberty?
The main runtime log is `messages.log`; application file logging can be configured separately through Logback.

72. How do you debug SQL activity?
Hibernate SQL and bind logging are enabled through `application.properties` log levels.

73. Why does the app support both Liberty and embedded Boot?
It gives local Java app-server deployment plus cloud/container flexibility.

74. What is the role of CORS config?
To allow the Angular frontend host to call backend APIs without browser blocking.

75. Why is the app stateless?
It uses JWT and no server-side session storage, which simplifies scaling.

Design tradeoffs and future work

76. Why was customer creation coupled with account opening?
It matches the current onboarding flow, but it could be split later if onboarding becomes multi-step.

77. Why is role data currently used only for access control?
Because the product has not yet added a UI/API to manage roles as a first-class admin feature.

78. What would be a good next queue use case?
Audit/event publishing for customer creation, role changes, and money-movement notifications.

79. What would be a good next persistence improvement?
A proper outbox table for Kafka events and a retry worker.

80. What would be a good next frontend improvement?
A real management roles page and listener/event monitoring page.

Backend deep dive

81. What does `CustomUserDetailsService` do?
It loads a user by username, fetches active roles, and adapts them into Spring Security authorities.

82. Why does it add `ROLE_` to role names?
Spring Security's role checks expect authorities in that format.

83. What does `JwtAuthenticationFilter` do?
It extracts a Bearer token, validates it, and populates the security context for the request.

84. Why is the JWT filter `OncePerRequestFilter`?
So the token logic runs once per request and not multiple times in the filter chain.

85. What happens when the auth header is missing?
The filter passes the request through and the security layer later decides whether access is allowed.

86. Why does the filter swallow exceptions?
Because unauthenticated requests should fail cleanly with the configured entry point, not with filter crashes.

87. What does `JwtAuthenticationEntryPoint` handle?
It returns the response for unauthenticated access attempts.

88. What does `JwtAccessDeniedHandler` handle?
It handles authenticated users who still lack required authorization.

89. Why is `SecurityConfig` using `SessionCreationPolicy.STATELESS`?
Because JWT auth doesn't need server sessions.

90. Why is CORS configured centrally?
So the Angular app can call the backend from allowed origins in browser environments.

91. Why allow `OPTIONS` requests?
Browser preflight requests need to succeed before real API calls can run.

92. Why permit `/auth/login` and `/error`?
Login must be public, and error handling should not be blocked by authentication.

93. Why are customer GET endpoints broader than POST/PUT/DELETE?
Different roles can view customers, but only staff should create or modify them.

94. Why are `/users/**` endpoints admin-only?
Employee administration is restricted to admins in this implementation.

95. What does `BusinessIdFormatter` solve?
It converts numeric ids into business-friendly display codes like employee codes.

96. Why is `GlobalExceptionHandler` important?
It keeps API errors consistent and prevents stack traces from leaking to clients.

97. What is the purpose of `ErrorResponse`?
It standardizes the structure of error replies.

98. Why use custom exceptions like `BadRequestException` and `ConflictException`?
They let the application express intent and map to proper HTTP status codes.

99. What does `PageRequests.of(...)` protect against?
Invalid sort fields and inconsistent sort direction inputs.

100. Why is `@Valid` used on request bodies?
To enforce DTO validation before business logic runs.

Data model questions

101. Why does `Account` belong to a `Customer`?
Accounts are customer-owned in this domain model.

102. Why is `UserRole` separate from `Role`?
It allows assignment history and active/inactive state per user-role link.

103. Why does `UserRole` need an `active` flag?
It lets the app deactivate old access levels without deleting history.

104. Why is `Role` unique on role name?
Because role names are business identifiers and should not duplicate.

105. Why is `AccountRepository.getNextOrdinal()` a native query?
Because it directly reads from a database sequence.

106. Why is `findByIdForUpdate` needed?
It acquires a pessimistic write lock for balance-sensitive operations.

107. Why are some repository methods derived from names?
Spring Data can implement them automatically, reducing boilerplate.

108. Why does `UserRepository.countEmployees()` use JPQL?
It counts distinct users with active employee-like roles across joined tables.

109. Why does dashboard employee count not just count all users?
Because only specific active roles represent bank staff.

110. Why is the dashboard transaction count zero?
The code currently stubs that metric and can be expanded later.

Notification and mail deep dive

111. What is `BankActionEvent` used for?
It carries account action metadata from the account service to the notification consumer.

112. Why send notifications asynchronously?
So account operations don't wait on SMTP/HTTP email delivery.

113. Why does the notification consumer compose email content separately?
It keeps business event data separate from the final mail template.

114. Why are there two mail implementations?
SMTP works locally; SendGrid works when outbound SMTP is blocked in hosting environments.

115. When is SendGrid used?
When `app.mail.transport=sendgrid`.

116. Why is `SendGridNotificationMailService` marked `@Primary`?
So it wins when the sendgrid condition is active.

117. Why does SMTP have a fallback recipient?
To avoid losing messages when customer email is unavailable.

118. Why does SendGrid use HTTPS instead of SMTP?
Cloud hosts often block SMTP ports, but HTTPS outbound is usually allowed.

119. What happens if mail sending fails?
The consumer logs the failure, and the current design does not retry automatically.

120. What is the business risk there?
An account action can be committed but the email may be lost.

Frontend deep dive

121. Why are standalone Angular components used?
They reduce module overhead and keep feature code self-contained.

122. Why does the frontend use signals?
Signals make local UI state and derived state easier to manage.

123. Why does the frontend still use RxJS?
HTTP and combined async streams are naturally handled by RxJS.

124. What does `toSignal()` do here?
It turns an observable stream into reactive template-friendly state.

125. Why are loading and error states repeated per page?
Each page owns its own API flow and needs responsive state handling.

126. Why does the list search debounce?
To reduce backend calls while the user is typing.

127. Why reset page index on search and sort changes?
Because the filtered/sorted result set should start from the first page.

128. Why do detail pages reload after edit dialogs close?
So the screen refreshes to reflect the updated data.

129. Why does the UI use router links for some actions and dialogs for others?
Navigation is used for full pages; dialogs are used for quick create/edit actions.

130. Why is access controlled in both backend and frontend?
Frontend improves UX, but backend is the actual security boundary.

131. What does `authInterceptor` add?
It attaches the Bearer token to outgoing API requests.

132. Why does the interceptor skip login requests?
Because the login request doesn't yet have a token.

133. Why does it logout on 401?
Because a rejected token means the session is no longer valid.

134. Why does the guard redirect unauthenticated users to `/`?
That is the login route in this app.

135. Why redirect unauthorized users to dashboard?
It keeps them inside the app shell instead of leaving them on a blank denied page.

136. How does profile data load?
It calls `/auth/me`, then renders identity, status, and role information.

137. Why does profile show `lastLogin`?
It helps users confirm their recent access history.

138. Why is the dashboard cached?
API (Redis, local): `DashboardServiceImpl` uses Spring `@Cacheable` (`dashboard` / `summary`, ~60s TTL) so summary counts are not counted from Postgres on every hit. Writes that change customers/accounts/users evict that region. See MODULES/Caching.md.

UI: the Angular dashboard may also avoid refetching on every navigation; logout clears the client session (and thus client-side cache).

139. When is dashboard cache cleared?
Redis: on related `@CacheEvict` (customer/account/user money or master changes) or when the 60s TTL expires.
UI: on logout / session clear; user can also use refresh if the screen provides it. **140. Why does dashboard refresh exist?**
It forces a new fetch for the current summary.

Deployment and configuration

141. How does the app run locally?
Backend on Liberty, frontend on Angular dev server, PostgreSQL locally or via Docker.

142. What does `DataSourceConfig` do?
It parses `DATABASE_URL`-style values and configures a Hikari datasource.

143. Why is Render-specific SSL handling present?
Some hosts require `sslmode=require`, so the config adds it automatically when needed.

144. Why support both `postgres://` and `jdbc:postgresql://`?
Different platforms expose different connection-string formats.

145. Why is auditing enabled?
So common created/updated timestamps are managed consistently.

146. Why does `AuditorConfig` return `1L`?
It is a temporary placeholder auditor, not a real user-aware implementation yet.

147. What should a better auditor do later?
Resolve the current authenticated user id from security context.

148. Why are logs configured through properties?
So SQL and web/security logging can be adjusted without code changes.

149. Why is logging split between Liberty and application logging?
Liberty captures process output, while Logback can write structured application logs separately.

150. Why is the backend packaged as WAR?
To deploy cleanly on Open Liberty.

Design and extension questions

151. Why would a transactional outbox help here?
It guarantees events are persisted with the database transaction before Kafka publishing.

152. Why is this useful for banking?
It reduces the chance of losing audit/notification events after money changes.

153. What would you queue next after notifications?
Audit events, reporting refreshes, and role/user lifecycle events.

154. What should never be queued?
Core balance mutation and validation decisions.

155. Why is a role-management UI a natural next step?
Roles already exist in the DB and drive access control, so a UI would be immediately useful.

156. Why would an event-monitor page be useful?
It would let admins see Kafka events, mail status, and retries from the app itself.

157. Why is branch support currently limited?
Branch code is used as a business string, not a full branch module yet.

158. Why is loan support partial?
Loan account type exists, but onboarding still blocks creating loan accounts directly.

159. Why does the code have so many “Soon” cards in management?
They mark planned admin features that are not built yet.

160. What is the best interview answer about the project's current state?
It's a working banking platform with auth, customers, accounts, transactions, notification events, and a growing Angular admin UI.

Deep dive walkthroughs

1) Login and authorization flow

When a user logs in, the frontend sends the username and password to `POST /auth/login`. The backend does not trust the request body by itself; `AuthenticationService` first looks up the user, checks whether the account is locked, and then delegates the credential check to Spring Security's `AuthenticationManager`. That manager uses `CustomUserDetailsService`, which loads the user and active roles from the database and exposes them as authorities. If authentication succeeds, `JwtService` generates a token, and the backend also updates `lastLogin` and resets failed attempts.

After login, the frontend stores the token and role list in browser storage. From that point on, `authInterceptor` attaches the token to every API call, and `authGuard` blocks routes when the user is either unauthenticated or does not have the route's allowed roles. That means security is enforced twice: once by the frontend for UX and once by the backend for real access control.

2) Customer onboarding flow

Customer onboarding is intentionally opinionated. The backend accepts one create-customer request, validates uniqueness for email and phone, saves the customer, and then immediately opens the first account. That makes onboarding feel like a business operation rather than a set of loosely coupled database inserts. The service also enforces a rule that loan accounts cannot be created during customer onboarding, which is a product decision rather than a technical limitation.

The important interview point here is that customer creation is not just CRUD. It triggers account-policy validation, account-number generation, and sometimes a ledger entry if the opening deposit is required and provided. That means the onboarding flow crosses customer, account, and transaction domains in one business transaction.

3) Account opening and policy flow

When an account is opened, the code first looks up the customer and then finds an active policy for the requested account type and currency. The policy matters because banking rules are not static: a current account may require a minimum opening deposit, and some policies can have effective date windows. If the policy says an opening deposit is mandatory, the service enforces that before any persistence happens.

Then the app obtains the next ordinal from the database sequence and combines branch code, account type, currency, and ordinal into a business account number. This is why `AccountNumberGenerator` exists: it keeps account numbers deterministic and business-friendly. Once the account is saved, the service optionally creates a transaction record for the opening deposit and publishes an account-opened notification event.

4) Deposit, withdraw, and transfer flow

Deposit, withdraw, and transfer all follow the same broad pattern: validate the input, load account(s), enforce business rules, update balances, save the entity, record transaction rows, and publish notification events. The difference is in the locking and invariants. Withdrawal locks the account row

because insufficient-funds checks must be based on the latest balance. Transfer locks both accounts in a stable order so two concurrent transfers do not deadlock each other.

In transfer, the source and destination accounts must be active and must use the same currency. That is a business rule the service enforces before any money moves. After updating balances, the service writes one debit transaction and one credit transaction so the ledger mirrors both sides of the movement. This is important in interviews because it shows the app treats account state and transaction history as separate but related records.

5) Kafka notification flow

After a banking event completes, the account service publishes a `BankActionEvent` to Kafka. That event is not the email itself; it is a compact business event describing what happened, which account or entity was affected, who acted, and which customer email should receive the notice. The consumer then receives the event, composes the subject and body, and sends the actual email through SMTP or SendGrid depending on the environment.

This separation matters because email sending is slow and failure-prone compared to database writes. If the app tried to send mail inline during the transfer or deposit transaction, users would see slower responses and a mail outage could break banking operations. By using Kafka, the app keeps the core money action fast and lets the side effect happen asynchronously.

6) Frontend state and list rendering flow

The Angular app uses a reactive pattern for most lists and detail screens. A user types into search, changes sorting, or changes page size, and those signals are merged with RxJS streams. The UI then issues a new API call and maps the result into `loading`, `loaded`, or `error` state. That gives the page a simple mental model and avoids manual subscription cleanup.

The interview angle here is that the UI is not static HTML. It reacts to state, and the state comes from backend paging APIs that already enforce allowed sort fields. That gives the frontend a responsive feel while preserving backend control over what can be queried.

7) Security and error-handling flow

Spring Security and the global exception handler work together. Security decides whether the request should proceed. If it should not, the JWT entry point or access-denied handler returns the appropriate response. If the request gets past security but fails business validation, the controller or service throws an application exception and `GlobalExceptionHandler` turns it into a structured error response.

This layering matters because it keeps auth failures distinct from business-rule failures. For example, a missing token should not look the same as an invalid deposit amount or a duplicate email. In interviews, this is a good place to explain how you keep concerns separate and make client error handling predictable.

8) Deployment and runtime flow

The backend is designed to run in two modes: Open Liberty as a WAR and embedded Boot for Docker/cloud. The same code reads datasource details from standard Spring properties or platform-specific environment variables. It also supports JDBC URLs and `postgres://`-style URLs, which makes deployment easier across local machines, Render, and containers.

At runtime, logs appear in Liberty's messages log and in application logs if Logback is enabled. Hibernate SQL/bind logging can be turned on through configuration, which makes debugging account and transaction behavior much easier. A strong interview answer here is that the codebase is built for both portability and observability.

Interview handbook: how to answer

Use this structure for most questions:

1. **What it is** — define the feature in one sentence.
2. **Why it exists** — explain the business need.
3. **How it works** — describe the path through controller/service/repository/UI.
4. **Edge cases** — call out validation, locking, auth, and failure behavior.
5. **Tradeoff** — mention what the code currently does well and what you would improve later.

Strong answer pattern

"This feature solves X because Y matters to the business. In the code, the controller receives the request, the service applies the rules, and the repository stores the result. Edge cases like invalid input, duplicates, or concurrent updates are handled before the data is committed. If I had to improve it, I'd add E."

Common interview traps

- Saying "Kafka is a queue" without explaining that it is a topic-based event stream in this code.
- Saying "JWT is just login" without mentioning stateless auth, Spring Security, and filters.
- Saying "deposit/withdraw is CRUD" without explaining validation, transactions, and ledger rows.
- Forgetting that security is enforced both in the backend and in the Angular route guard.
- Ignoring concurrency concerns in transfer/withdraw.

Short memorisable answer style

For fast rounds, answer in this order: - one-line definition - one-line reason - one-line flow - one-line edge case

Topic playbooks

Authentication and authorization

What JWT-based stateless authentication backed by Spring Security. Login creates a token, and each request carries the token in the Authorization header.

Why The app needs secure access control without server sessions. Stateless auth is simpler for frontend apps and easier to scale.

How `AuthenticationController -> AuthenticationService -> AuthenticationManager -> CustomUserDetailsService -> JwtService`. The filter reads the token later and repopulates the security context.

Edge cases Wrong password, locked account, missing token, expired token, and role mismatch.

Common trap Explaining JWT only as "token login" and forgetting the filter chain and authorities.

Model answer “Login is JWT-based and stateless. The backend authenticates credentials through Spring Security, then returns a token. On later requests, the JWT filter validates that token and loads the user authorities, which lets `@PreAuthorize` and URL rules enforce access.”

Customer onboarding

What A new customer record is created, uniqueness is checked, and the first account is opened as part of onboarding.

Why The product treats onboarding as a business process, not as a simple customer insert.

How `CustomerController -> CustomerServiceImpl.createCustomer -> CustomerRepository.save -> AccountService.openAccount`.

Edge cases Duplicate email/phone, missing data, invalid account type, loan accounts blocked during onboarding.

Common trap Forgetting that onboarding also triggers account creation and sometimes a ledger event.

Model answer “Customer onboarding is handled in one service method: validate uniqueness, persist the customer, then create the initial account. That keeps the first banking relationship consistent and avoids leaving a customer half-created without banking access.”

Accounts and policies

What Accounts belong to customers and are governed by active account policies by type and currency.

Why Banking rules differ by account type and need policy-driven enforcement.

How The service loads the customer, loads the active policy, enforces effective dates and minimum deposit rules, generates the account number, saves the account, and optionally writes an opening transaction.

Edge cases No active policy, policy not yet effective, expired policy, required opening deposit missing.

Common trap Saying the account number is random. It is business-generated and sequence-based.

Model answer “Account opening is policy-driven. The service first verifies that the policy is active and valid for the requested type and currency, then generates a deterministic account number and saves the account. The policy acts as the business gatekeeper.”

Transactions and money movement

What Deposit, withdraw, and transfer update balances and store immutable transaction history.

Why Money movement must be auditable and resistant to race conditions.

How Validate request, lock account(s) where needed, check status/currency/funds, mutate balances, save, then write transaction rows.

Edge cases Insufficient funds, inactive accounts, same-account transfer, currency mismatch, concurrent withdrawal.

Common trap Forgetting the ledger rows and talking only about the account balance.

Model answer “The balance change and the ledger entry are separate steps. The account table shows the current state, while the transaction table keeps the history. For withdrawals and transfers, the code locks rows to avoid concurrent overspending.”

Kafka notifications

What Bank actions publish events to Kafka, and a consumer turns them into customer email notifications.

Why Emails are slow and failure-prone compared to database writes, so they should not block core banking actions.

How `AccountServiceImpl` publishes a `BankActionEvent`, `NotificationEventConsumer` receives it, `NotificationEmailComposer` builds the mail, and `NotificationMailService` sends it.

Edge cases No recipient email, Kafka unavailable, mail provider failure, duplicate events, and event ordering.

Common trap Thinking Kafka is the business operation itself. In this code it is only the async side effect path.

Model answer “Kafka is used as the async event path for notifications. The banking operation commits in the database first, and then an event triggers the email flow. That keeps core banking fast and isolates mail delivery from the transaction.”

Frontend architecture

What Angular standalone components, route guards, signals, and reusable list components make up the UI.

Why The frontend needs a modular structure that is easy to extend by feature.

How Pages call services, services call REST APIs, signals track loading/error state, and route data controls access.

Edge cases Unauthorized redirects, loading/error states, stale cached dashboard data, list pagination resets.

Common trap Explaining the frontend as if it is just templates; the state flow matters a lot here.

Model answer “The UI is feature-based Angular. Each page owns its API stream and uses signals for derived state. Access is protected both by route guards and by backend authorization, so UI restrictions are only the first layer.”

Error handling and observability

What The app maps exceptions into structured error responses and logs through Liberty plus application logging.

Why Users need stable API errors and developers need traceable server behavior.

How Business exceptions bubble up to `GlobalExceptionHandler`, while Liberty logs and Hibernate SQL logs help diagnose runtime issues.

Edge cases Duplicate data, invalid input, locked account, permission denied, and backend startup issues.

Common trap Treating a 500 the same as a validation problem.

Model answer “Validation and business errors are intentionally separated from security failures and runtime exceptions. That makes the API predictable and easier to support, especially in banking flows where the difference between conflict, bad request, and unauthorized matters.”

Follow-up drill questions and answers

Authentication drill

Q: Why do you need both `authGuard` and backend `@PreAuthorize`?
`authGuard` improves UX by hiding pages users cannot access, but it is not security by itself. A user can still call backend APIs directly from tools like Postman. The backend must therefore enforce the actual rule. That is why the code uses `@PreAuthorize` on controllers and also checks roles in the Angular route config.

Q: What would break if the JWT filter were removed?
Requests would no longer have their security context rebuilt from the token, so the backend would treat authenticated users as anonymous on each request. That would make `@PreAuthorize` fail and protected endpoints would return 401/403 even if the frontend sent a token.

Q: Why is `ROLE_` prefixed in `CustomUserDetailsService`?
Spring Security’s role checks are built around `ROLE_`-prefixed authorities. The database stores plain names like `ADMIN`, but Spring expects `ROLE_ADMIN` when evaluating `hasRole('ADMIN')`.

Customer drill

Q: Why not just save the customer and let the UI open an account later?
Because the product flow treats onboarding as one business action. If the customer is created but the initial account is not opened, the onboarding experience is incomplete and the business has to handle partial state. Doing both together reduces that gap.

Q: How does the service protect customer data quality?
It trims fields, checks duplicate email and phone, validates inputs through request DTOs, and rejects invalid onboarding steps like loan account creation. The service is enforcing business rules before data reaches the database.

Account drill

Q: What is the difference between business identity and technical identity for an account?
The technical identity is the database `accountId`. The business identity is the generated account number, branch code, account type, currency, and ordinal. Interviews often expect you to know both because users see the account number while the DB uses the numeric id.

Q: Why do you need a policy table at all?
Because account opening rules should not be hardcoded. Minimum opening deposit, effective date, and currency/type combinations can change over time. Putting them in a policy table makes the business rules configurable rather than baked into Java constants.

Transaction drill

Q: Why keep a separate transaction table if the account balance is already updated?
Because balances tell you the current state, but transactions tell you how you got there. In banking,

history is just as important as the current number. Without a ledger, you cannot explain why the balance changed or audit an error later.

Q: Why does withdrawal lock the row?
Without a lock, two concurrent withdrawals could both read the same available balance and both succeed, leaving the account overdrawn. The pessimistic lock makes the read-modify-write sequence safe.

Q: Why is transfer more complicated than deposit?
Deposit updates one account only. Transfer touches two accounts, must preserve money conservation, must avoid deadlock, and must write two transaction rows. It has more places to fail and more invariants to preserve.

Kafka drill

Q: Why not send email inside the same account transaction?
If email sending is inside the transaction, the user waits on external infrastructure and a mail failure could roll back the banking action or at least make it slower and more fragile. Kafka decouples those concerns.

Q: What is the biggest weakness in the current Kafka design?
The event is published after the banking action, but there is no transactional outbox or retry state. If Kafka is down after commit, the email event can be lost. Banking systems usually want stronger guarantees than “log and continue”.

Frontend drill

Q: Why use `toSignal()` on top of observables?
Because Angular templates are easier to bind to signals than to manually subscribed observables. It also fits the component style used throughout the app.

Q: Why reset page number on search?
If you search while on page 7, the filtered dataset might only have 1 or 2 pages. Resetting to page 0 avoids empty or confusing results.

Q: Why is the dashboard cached?
Dashboard counts are summary data. BankOne caches them in **Redis** (cache-aside, short TTL) and the UI may avoid refetching on every navigation. Eviction / refresh keeps numbers honest after creates and money moves. Details: MODULES/Caching.md.

Deployment drill

Q: Why support both `jdbc:postgresql://` and `postgres://` URLs?
Different hosting platforms expose connection strings in different formats. Supporting both avoids deployment-specific code changes.

Q: Why is `sslmode=require` added automatically on some hosts?
Some managed Postgres platforms require SSL for connectivity. The config detects those hosts and appends the setting so the app works out of the box.

5-minute revision sheet

If you only have 5 minutes, remember this:

- **Auth**: JWT + Spring Security + stateless + `@PreAuthorize` + frontend guard
- **Customer**: create customer, enforce uniqueness, open first account
- **Account**: policy-driven opening, account number generation, status updates
- **Transaction**: deposit/withdraw/transfer, locks, ledger rows, audit history
- **Kafka**: async notifications only, not core balance logic
- **Frontend**: standalone Angular, signals, RxJS, route guards, dialogs
- **Deployment**: Liberty WAR, embedded Boot support, PostgreSQL config, logs in Liberty

One-line memorisable summary

“BankOne is a stateless JWT-secured banking platform where the backend owns business rules, the frontend owns UX, account flows are policy-driven, money movement is transactional and auditable, and Kafka handles asynchronous notifications.”

Good interview follow-up questions

Q: Why did you choose JWT instead of server sessions?
JWT fits this app because the frontend is a separate Angular SPA and the backend is an API service. Stateless tokens keep the backend simpler to scale, avoid server-side session storage, and work well with the route-guard plus interceptor model already used in the UI.

Q: Why did you keep balance updates synchronous but notifications async?
Because the balance change is the business-critical action. It must succeed or fail inside the database transaction. Notifications are important, but they are secondary side effects, so they should not slow down or endanger the money movement itself.

Q: How would you add a new role-management screen safely?
I would first expose a read-only `/roles` API, then build a frontend list page for admins. After that, I would add a controlled edit flow only if the backend has proper role assignment rules, validation, and audit logging. Security must stay server-side; the UI alone is not enough.

Q: How would you make notification delivery reliable across Kafka failures?
I would use a transactional outbox. The account transaction would write both the business change and an outbox record in the same database transaction. A separate publisher would read unsent outbox rows and publish them to Kafka with retries and idempotency.

Q: How would you add audit history for money movement?
The current `Transaction` table is already a strong starting point, but I would add richer audit metadata such as request id, actor id, channel, correlation id, and before/after balance snapshots. That would make investigations and compliance reporting much easier.

Q: How would you introduce a transactional outbox here?
I would create an `outbox_event` table with payload, event type, aggregate id, status, retry count, and timestamps. The account service would write the outbox row in the same transaction as the account update, and a background worker would deliver it to Kafka and mark it as sent.

Q: How would you support branch-level permissions later?
I would extend the current role model with branch scope or branch assignment data, then enforce it in service methods and `@PreAuthorize` checks. The UI could hide branch-inaccessible data, but the backend would need to verify the user's branch context before returning or mutating records.

Q: How would you prevent duplicate onboarding requests?
I would keep the uniqueness checks on email and phone, but I would also add idempotency for onboarding requests, probably through a client-generated request id or a server-side deduplication key. That protects against double-clicks and retry storms where the same request is submitted twice.